

Parallel Multiobjective Decision Diagrams

Anonymous submission

Abstract

Decision diagrams (DDs) can efficiently enumerate the exact Pareto frontier (PF) for multiobjective discrete optimization problems with a compact dynamic programming formulation. However, existing DD-based PF enumeration algorithms are serial, presenting significant opportunities for parallelization. In this work, we identify the serial operations amenable to parallelization and develop efficient parallel implementations to accelerate enumeration on both CPU and GPU. Finally, we evaluate our methods on benchmark instances of the multiobjective knapsack, set packing, and traveling salesperson problems with up to seven objectives, comparing against a state-of-the-art MIP approach and the state-of-the-art serial DD implementation. DD-based enumeration proves more robust than the MIP baseline on MOKP and MOTSP, though the MIP baseline remains competitive on some lower-objective MOSPP instances. Relative to serial DD enumeration, our parallel CPU variants achieve speedups of up to $7.83\times$, while GPU variants reach up to $20.88\times$ when intermediate frontiers fit in GPU memory. Code is available at <https://anonymous.4open.science/r/DD-C683/>.

1 Introduction

Many discrete optimization problems that arise in practice are inherently multiobjective. In vehicle routing, for example, a dispatcher does not just minimize routing cost. Service quality also depends on how consistently each customer is visited by the same driver and at a similar time each day, since both build trust and route familiarity (Kovacs, Paragh, and Hartl 2015). These goals conflict, since restricting driver assignments or tightening arrival-time windows typically forces costlier routes, so no single routing plan is best on cost, driver consistency, and arrival-time consistency at once. Such plans cannot be ranked by a single score; they are compared through Pareto dominance, where one solution dominates another if it matches it on every objective and improves on at least one. The goal of exact multiobjective optimization is to enumerate every nondominated solution, since each one represents a distinct trade-off a decision-maker might reasonably prefer over the rest. This complete collection is known as the *Pareto frontier* (PF). It exposes every trade-off among the objectives without approximation and leaves the final choice to the decision-maker.

Multiobjective discrete optimization (MODO) (Ehrgott 2005; Ehrgott, Gandibleux, and Przybylski 2016; Miettinen 1999)

problems of this kind are common in routing, scheduling, and resource-allocation settings, among others. Problem-specific heuristics (Vianna and Arroyo 2004) and evolutionary methods (Deb et al. 2002) are widely used to obtain a representative subset of the frontier quickly, but they cannot certify that every trade-off has been found; exact enumeration of the PF remains necessary whenever this certification is required. Decision diagrams (DDs) (Bergman et al. 2016) provide an effective exact framework for discrete problems with a dynamic programming structure. An exact DD compactly represents the feasible set as a layered acyclic graph; PF enumeration can then be performed by propagating nondominated labels through this graph. Recent DD-based methods have shown strong performance on MODO problems with several objectives and tens to hundreds of decision variables (Bergman and Cire 2016; Bergman et al. 2022).

A DD by itself does not make PF enumeration cheap. Every DD node stores a nondominated set of labels for the partial paths that reach it, and expanding a layer generates new candidate labels, merges them into the destination node’s frontier, and discards the ones some other label dominates. This is expensive because dominance filtering compares every candidate against every other candidate at a node. Enumeration also works inward from both ends of the diagram, alternating between expanding forward from the root and backward from the terminal until the two meet at a shared middle layer called the *cutset*, where the root-side and terminal-side frontiers are combined and filtered once more. Existing DD-based MODO solvers perform all of this one node, one comparison at a time.

This paper studies how to parallelize exact DD-based PF enumeration. The key observation is that the expensive work in bidirectional PF enumeration decomposes at multiple granularities. Layer expansion can be distributed over nodes; candidate generation and dominance checks can be distributed over labels; and the final cutset merge can be organized as a parallel reduction over nondominated sets. These forms of parallelism map differently to multi-core CPUs and GPUs. CPUs are effective for node-level work sharing and tree-structured frontier reduction, whereas GPUs can expose finer-grained parallelism by materializing candidates in flat buffers and filtering them with data-parallel kernels.

We make the following contributions:

- We identify where bidirectional PF enumeration decomposes for parallel execution. Specifically, across nodes within a layer, across labels within a node, and across products and merges in the coupling phase.
- We build a CPU implementation that dynamically schedules node expansion over OpenMP threads and reduces cutset frontiers with a lock-free pairwise tree merge.
- We develop a GPU implementation that flattens the diagram into offset-indexed arc arrays, expands each layer with a four-kernel pipeline, and merges the cutset in memory-bounded batches against a running frontier.
- We evaluate on MOKP, MOSPP, and MOTSP instances with up to seven objectives against a state-of-the-art MIP baseline (Dächert, Fleuren, and Klamroth 2024) and the state-of-the-art serial DD baseline (Bergman et al. 2022), and show that our parallel CPU and GPU variants cut enumeration time relative to serial DD by up to $7.83\times$ and $20.88\times$ respectively, with the GPU fastest when frontiers fit in device memory and the CPU more reliable when they don't.

The remainder of this paper is organized as follows. Section 2 provides background on MODO and DDs. Section 3 describes the parallel enumeration methodology. Section 4 presents the computational evaluation. Section 5 reviews related work. Section 6 concludes the paper.

2 Preliminaries

2.1 Multiobjective Discrete Optimization

A MODO problem can be defined as the minimization of an m -dimensional objective vector function $\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$ subject to $\mathbf{x} \in \mathcal{X}$. Here, $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ represents a vector of discrete decision variables, and $\mathcal{X} \subseteq \mathbb{Z}^n$ is the bounded, discrete feasible region. We compare different feasible solutions based on the principle of *Pareto dominance*. For any two feasible solutions $\mathbf{x}^1, \mathbf{x}^2 \in \mathcal{X}$, the solution $\mathbf{x}^1$ is said to Pareto dominate $\mathbf{x}^2$, denoted by $\mathbf{f}(\mathbf{x}^1) \prec \mathbf{f}(\mathbf{x}^2)$, if $f_i(\mathbf{x}^1) \leq f_i(\mathbf{x}^2)$ for all $i \in \{1, \dots, m\}$, and there exists at least one index $j \in \{1, \dots, m\}$ strictly satisfying $f_j(\mathbf{x}^1) < f_j(\mathbf{x}^2)$. An objective vector $\mathbf{y}^* = \mathbf{f}(\mathbf{x}^*)$ associated with a feasible solution $\mathbf{x}^* \in \mathcal{X}$ is defined as a *nondominated point* if there exists no other feasible solution $\mathbf{x} \in \mathcal{X}$ such that $\mathbf{f}(\mathbf{x}) \prec \mathbf{y}^*$. The collection of all such nondominated points across the entire feasible region constitutes the *PF* (or non-dominated frontier) of the MODO problem. Mathematically, this frontier is expressed as $\mathcal{Y}_N = \{\mathbf{f}(\mathbf{x}) \mid \mathbf{x} \in \mathcal{X}, \nexists \mathbf{x}' \in \mathcal{X} \text{ such that } \mathbf{f}(\mathbf{x}') \prec \mathbf{f}(\mathbf{x})\}$. The fundamental aim in exact multiobjective optimization is to completely enumerate this PF, providing decision-makers with the exact trade-offs among the competing criteria.

2.2 Decision Diagrams for Multiobjective Discrete Optimization

A DD is a directed layered acyclic graph $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ defined by a node set $\mathcal{N}$ and an arc set $\mathcal{A}$. The node set is partitioned into $n + 1$ strictly sequential layers, $\mathcal{N} =$

Algorithm 1 Bidirectional PF Enumeration

Require: Exact decision diagram $\mathcal{D} = (\mathcal{N}, \mathcal{A})$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```

1: procedure FILTERDOMINATED( $\mathcal{Y}$ )  $\triangleright \mathcal{O}(|\mathcal{Y}|^2)$ 
2:   return  $\mathcal{Y} \leftarrow \{\mathbf{y} \in \mathcal{Y} \mid \nexists \mathbf{y}' \in \mathcal{Y} \text{ such that } \mathbf{y}' \prec \mathbf{y}\}$ 
3: end procedure
4: procedure EXPANDTOPDOWN( $j, \mathcal{Y}^\downarrow$ )
5:   for each node  $u \in \mathcal{L}_j$  do
6:     for each arc  $a = (u, v)$  originating from  $u$  do
7:        $\mathcal{Y}^\downarrow(v) \leftarrow \mathcal{Y}^\downarrow(v) \cup (\mathcal{Y}^\downarrow(u) \oplus \{\mathbf{v}(a)\})$ 
8:        $\mathcal{Y}^\downarrow(v) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\downarrow(v))$ 
9:     end for
10:  end for
11:  return  $\mathcal{Y}^\downarrow$ 
12: end procedure
13: procedure EXPANDBOTTOMUP( $j, \mathcal{Y}^\uparrow$ )
14:   for each node  $v \in \mathcal{L}_j$  do
15:     for each arc  $a = (u, v)$  entering  $v$  from  $u \in \mathcal{L}_{j-1}$  do
16:        $\mathcal{Y}^\uparrow(u) \leftarrow \mathcal{Y}^\uparrow(u) \cup (\mathcal{Y}^\uparrow(v) \oplus \{\mathbf{v}(a)\})$ 
17:        $\mathcal{Y}^\uparrow(u) \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}^\uparrow(u))$ 
18:     end for
19:   end for
20:   return  $\mathcal{Y}^\uparrow$ 
21: end procedure
22:  $\mathcal{Y}^\downarrow(u) \leftarrow \emptyset, \mathcal{Y}^\uparrow(u) \leftarrow \emptyset$  for all  $u \in \mathcal{N}$ 
23:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}, \mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{\mathbf{0}\}$   $\triangleright \mathbf{r}$  is root,  $\mathbf{t}$  is terminal
24:  $i_{\text{td}} \leftarrow 1, i_{\text{bu}} \leftarrow n + 1$ 
25: while  $i_{\text{td}} \neq i_{\text{bu}}$  do
26:   Dynamically decide whether to expand top-down or bottom-up
27:   if expanding top-down then
28:     EXPANDTOPDOWN( $i_{\text{td}}, \mathcal{Y}^\downarrow$ ),  $i_{\text{td}} \leftarrow i_{\text{td}} + 1$ 
29:   else
30:     EXPANDBOTTOMUP( $i_{\text{bu}}, \mathcal{Y}^\uparrow$ ),  $i_{\text{bu}} \leftarrow i_{\text{bu}} - 1$ 
31:   end if
32: end while
33:  $c \leftarrow i_{\text{td}}, \mathcal{Y}_N \leftarrow \emptyset$ 
34: for each node  $u \in \mathcal{L}_c$  do
35:    $\mathcal{Y}(u) \leftarrow \mathcal{Y}^\downarrow(u) \oplus \mathcal{Y}^\uparrow(u)$   $\triangleright \mathcal{O}(|\mathcal{Y}^\downarrow(u)| |\mathcal{Y}^\uparrow(u)|)$ 
36: end for
37: for each node  $u \in \mathcal{L}_c$  do
38:    $\mathcal{Y}_N \leftarrow \text{FILTERDOMINATED}(\mathcal{Y}_N \cup \mathcal{Y}(u))$ 
39: end for
40: return  $\mathcal{Y}_N$ 

```

$\mathcal{L}_1 \cup \dots \cup \mathcal{L}_{n+1}$, corresponding to the sequence of the n decision variables. The first layer $\mathcal{L}_1$ contains only the *root* node $\mathbf{r}$, and the final layer $\mathcal{L}_{n+1}$ contains only the *terminal* node $\mathbf{t}$.

The arcs in $\mathcal{A}$ are directed from nodes in layer $\mathcal{L}_j$ to nodes in layer $\mathcal{L}_{j+1}$ for $j \in \{1, \dots, n\}$. Every arc $a \in \mathcal{A}$ originating from layer $\mathcal{L}_j$ possesses two key attributes: an *assignment value* $d(a) \in \mathbb{Z}$, which dictates the value assigned to the decision variable x_j , and an *objective weight* $\mathbf{v}(a) \in \mathbb{R}^m$, which represents the transition's contribution to the m -dimensional objective function. A path $p = (a_1, \dots, a_n)$

from the root $\mathbf{r}$ to the terminal node $\mathbf{t}$ encodes a complete decision vector $\mathbf{x}(p) = (d(a_1), \dots, d(a_n))^T \in \mathbb{Z}^n$, with an associated objective vector obtained by accumulating the arc weights along the path $\mathbf{v}(p) = \sum_{i=1}^n \mathbf{v}(a_i)$.

Let $\mathcal{X}_{\mathcal{D}} = \{\mathbf{x}(p) \mid p \text{ is an } \mathbf{r} - \mathbf{t} \text{ path in } \mathcal{D}\}$ denote the set of solutions encoded by the diagram. A DD is said to be *exact* for a given problem if it represents the feasible region ($\mathcal{X}_{\mathcal{D}} = \mathcal{X}$) and the path evaluations correctly compute the objective function ($\mathbf{v}(p) = \mathbf{f}(\mathbf{x}(p))$ for all paths). In this case, the image of the encoded solutions in the objective space is $\mathcal{Y}_{\mathcal{D}} = \{\mathbf{v}(p) \mid p \text{ is an } \mathbf{r} - \mathbf{t} \text{ path}\}$. The nondominated vectors extracted from $\mathcal{Y}_{\mathcal{D}}$ constitute the exact PF $\mathcal{Y}_N$ of the original MODO problem.

During enumeration, each DD node stores a *node frontier*: a nondominated set of objective vectors associated with partial paths through that node. In the root-side direction, $\mathcal{Y}^\downarrow(u)$ denotes the node frontier of partial paths from the root $\mathbf{r}$ to node u ; in the terminal-side direction, $\mathcal{Y}^\uparrow(u)$ denotes the node frontier of partial paths from u to the terminal $\mathbf{t}$. Individual objective vectors in a node frontier are called *labels*. Newly generated, unfiltered labels are called *candidate labels*. During cutset merging, a *global frontier* denotes a running nondominated set over complete root-to-terminal labels. We reserve *PF* for the final nondominated set $\mathcal{Y}_N$.

To extract this exact PF $\mathcal{Y}_N$ from an exact decision diagram $\mathcal{D}$, we can employ *bidirectional PF enumeration*, which dynamically interleaves root-side and terminal-side layer expansion, as detailed in Algorithm 1. This dynamic layer cutset methodology mitigates intermediate node-frontier growth. Initially, a root-side layer pointer i_{td} is set to layer 1 and a terminal-side pointer i_{bu} is set to layer $n+1$. At each step, the scheme dynamically decides whether to expand the root-side node frontiers $\mathcal{Y}^\downarrow$ to layer $i_{\text{td}} + 1$ or the terminal-side node frontiers $\mathcal{Y}^\uparrow$ to layer $i_{\text{bu}} - 1$. This process continues until the two directions meet at a *cutset* layer c (i.e., $i_{\text{td}} = i_{\text{bu}}$). Finally, for each node $u \in \mathcal{L}_c$, the algorithm computes the pairwise sums of labels in its root-side and terminal-side node frontiers (Algorithm 1, line 35). The merging of these combined sets and the repeated application of a dominance filter produce the exact PF $\mathcal{Y}_N$.

3 Parallel Enumeration of Pareto Frontier using Decision Diagrams

This section describes how we parallelize exact Pareto frontier enumeration on a decision diagram (DD), following the bidirectional PF enumeration scheme in Algorithm 1. The rule that chooses which side of the cutset to expand is inherited from the serial enumeration method. Our contribution is the parallel execution of the expensive primitives inside the enumeration: candidate generation, dominance filtering, cutset-product materialization, and frontier merging.

3.1 Sources of Parallelism

The opportunities for parallelism are most easily seen directly from Algorithm 1. In the root-side expansion, the loop over nodes in line 5 can be parallelized because the node frontier of each destination node is updated independently

once the previous layer has been finalized. The same observation applies to the terminal-side expansion in line 14. The work performed inside a node consists of scanning adjacent arcs, generating candidate labels, merging them into the node frontier, and applying the dominance filter in lines 7 and 17.

There is a second, finer level of parallelism inside lines 7 and 17. The candidate labels generated for a node can be materialized independently, and the pairwise dominance comparisons among those candidate labels can also be distributed. This finer decomposition requires an explicit candidate buffer and a careful layout so that different workers write to disjoint locations. It is therefore a different implementation choice from assigning one worker to own an entire node frontier.

The cutset phase exposes a separate form of parallelism. In lines 31–33, each cutset node $u \in \mathcal{L}_c$ contributes the product set

$$\mathcal{Y}^\downarrow(u) \oplus \mathcal{Y}^\uparrow(u) = \{\mathbf{y}^\downarrow + \mathbf{y}^\uparrow \mid \mathbf{y}^\downarrow \in \mathcal{Y}^\downarrow(u), \mathbf{y}^\uparrow \in \mathcal{Y}^\uparrow(u)\}.$$

These products are independent across cutset nodes and across pairs of labels within a node. The final merge in lines 34–36 is different: it is a reduction over nondominated sets. Node frontiers produced at the cutset must be combined while discarding points dominated by already retained points.

3.2 CPU Implementation

The CPU bidirectional PF enumeration implementation parallelizes the node loops in Algorithm 1. For root-side expansion, this is the loop in line 5; for terminal-side expansion, it is the loop in line 14. The loop is scheduled dynamically because the number of candidate labels can vary substantially across nodes in the same layer.

Each CPU worker owns the node frontier of the node it is assigned. It scans the relevant arcs, generates candidate labels, merges them into the node frontier, and applies the dominance filter corresponding to lines 7 and 17. Since no other worker mutates that node frontier, the implementation avoids locks. The limitation is that a single large node remains serial within one worker; the CPU implementation does not split the dominance filtering of one node across multiple workers.

The CPU also parallelizes the cutset products in lines 31–33. Each worker processes a subset of cutset nodes and builds a private cutset frontier from the products $\mathcal{Y}^\downarrow(u) \oplus \mathcal{Y}^\uparrow(u)$. The final merge in lines 34–36 is implemented as the pairwise tree reduction shown in Algorithm 4. This exposes parallelism in the global merge while keeping each node-frontier update thread-local.

3.3 GPU Implementation

The GPU implementation follows the same enumeration logic as Algorithm 1, but changes the unit of work. Rather than assigning one worker to own a node frontier, it materializes candidate labels in flat buffers and then applies data-parallel filtering and compaction kernels. The high-level procedure is shown in Algorithm 2: the DD is first flat-

Algorithm 2 GPU Bidirectional PF Enumeration

Require: DD $\mathcal{D}$ with layers $\mathcal{L}_1, \dots, \mathcal{L}_{n+1}$

```
1: procedure FLATTENDD( $\mathcal{D}$ )
2:   for  $j = 1$  to  $n + 1$  do
3:     // Top-down: incoming arcs of  $\mathcal{L}_j$ 
4:     Initialize source-node index per arc  $b^\downarrow$ 
5:     Initialize arc weight vector per arc  $\mathbf{v}^\downarrow$ 
6:     Initialize per-node incoming-arc offset  $\phi^{a,\downarrow}$ 
7:
8:     // Bottom-up: outgoing arcs of  $\mathcal{L}_j$ 
9:     Initialize destination-node index per arc  $b^\uparrow$ 
10:    Initialize arc weight vector per arc  $\mathbf{v}^\uparrow$ 
11:    Initialize per-node outgoing-arc offset  $\phi^{a,\uparrow}$ 
12:
13:    // Scoring heuristic
14:    Initialize out-degree per node  $\delta^{\text{out}}$ 
15:    Initialize in-degree per node  $\delta^{\text{in}}$ 
16:  end for
17:  Transfer all packed arrays to GPU global memory
18: end procedure
19: procedure EXPANDLAYER( $\mathcal{Y}, j, d$ )
20:   if  $d = \downarrow$  then
21:      $(b, \mathbf{v}, \phi^a) \leftarrow (b^\downarrow, \mathbf{v}^\downarrow, \phi^{a,\downarrow})$ 
22:   else
23:      $(b, \mathbf{v}, \phi^a) \leftarrow (b^\uparrow, \mathbf{v}^\uparrow, \phi^{a,\uparrow})$ 
24:   end if
25:    $\kappa, \phi^c, \Gamma \leftarrow \text{COMPUTEARCOFFSETS}(\mathcal{Y}, b, \phi^a)$ 
26:    $\beta \leftarrow \text{EXPANDCANDIDATES}(\mathcal{Y}, b, \mathbf{v}, \phi^c, \kappa)$ 
27:    $\sigma \leftarrow \text{FILTERDOMINATED}(\beta, \Gamma)$ 
28:    $\mathcal{Y} \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$ 
29:   return  $\mathcal{Y}$ 
30: end procedure
31: FLATTENDD( $\mathcal{D}$ )
32: Initialize on GPU:  $\mathcal{Y}^\downarrow(\mathbf{r}) \leftarrow \{\mathbf{0}\}, \mathcal{Y}^\uparrow(\mathbf{t}) \leftarrow \{\mathbf{0}\}$ 
33:  $i_{\text{td}} \leftarrow 1; \quad i_{\text{bu}} \leftarrow n + 1$ 
34:  $s_{\text{td}} \leftarrow 0; \quad s_{\text{bu}} \leftarrow 0$ 
35: while  $i_{\text{td}} \neq i_{\text{bu}}$  do
36:   if  $s_{\text{td}} \leq s_{\text{bu}}$  then ▷ Expand top-down
37:      $i_{\text{td}} \leftarrow i_{\text{td}} + 1$ 
38:      $\mathcal{Y}^\downarrow \leftarrow \text{EXPANDLAYER}(\mathcal{Y}^\downarrow, i_{\text{td}}, \downarrow)$ 
39:      $s_{\text{td}} \leftarrow \text{COMPUTESCORE}(\mathcal{Y}^\downarrow, i_{\text{td}})$ 
40:   else ▷ Expand bottom-up
41:      $i_{\text{bu}} \leftarrow i_{\text{bu}} - 1$ 
42:      $\mathcal{Y}^\uparrow \leftarrow \text{EXPANDLAYER}(\mathcal{Y}^\uparrow, i_{\text{bu}}, \uparrow)$ 
43:      $s_{\text{bu}} \leftarrow 1.5 \times \text{COMPUTESCORE}(\mathcal{Y}^\uparrow, i_{\text{bu}})$ 
44:   end if
45: end while
46:  $c \leftarrow i_{\text{td}}$  ▷ Cutset layer
47:  $\mathcal{Y}_N \leftarrow \text{CUTSETMERGE}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{L}_c)$ 
48: return  $\mathcal{Y}_N$ 
```

tened, each selected layer expansion invokes the same four-kernel pipeline, and the cutset phase is handled by a separate batched cutset-merge routine.

The flattened representation stores, for each layer and traversal direction, the relevant source or destination node index for each arc, the arc weight vector, and a per-node

arc-offset array. This layout makes each layer expansion a segmented operation over contiguous arc and candidate arrays. The representation is built once and reused throughout the enumeration.

For a selected layer, Algorithm 2 calls the four subroutines in Algorithms 5 to 7 and 9. The dataflow is

$$\mathcal{Y} \rightarrow (\kappa, \phi^c, \Gamma) \rightarrow \beta \rightarrow \sigma \rightarrow \mathcal{Y}_{\text{next}}.$$

Here κ_a is the number of candidate labels generated by arc a , ϕ_a^c is the starting offset of arc a in the flat candidate buffer, and Γ_v is the total number of candidate labels assigned to destination node v . The buffer β stores the generated candidate labels, and the binary vector σ marks the labels that survive node-frontier dominance filtering.

First, Algorithm 5 counts how many candidate labels each arc will produce and uses a prefix sum to assign disjoint output segments in β . Second, Algorithm 6 fills this buffer with shifted labels. The prefix-sum offsets are the key invariant: candidate labels generated by different arcs write to disjoint memory, but candidate labels for the same destination node occupy one contiguous segment. Third, Algorithm 9 filters each destination-node segment using a load-balanced grid of thread blocks; nodes with larger candidate label sets receive more blocks. Fourth, Algorithm 7 compacts the surviving labels into the next node-frontier representation. These four steps implement the parallelization of candidate generation, merging, and dominance filtering in lines 7 and 17 of Algorithm 1.

The cutset phase uses a different schedule, shown in Algorithm 3. Active cutset nodes are those with nonempty root-side and terminal-side node frontiers. They are sorted by node-frontier size and processed in product-bounded batches so that the Cartesian product for a batch fits in memory. For each batch, Algorithm 8 materializes all sums $\mathbf{y}^\downarrow + \mathbf{y}^\uparrow$ corresponding to lines 31–33 of Algorithm 1. The batch is then filtered against the current global frontier, self-pruned, and merged into the running global frontier. Thus the GPU does not use the CPU tree reduction in Algorithm 4; it uses a batched running-frontier merge, where batching is both a load-balancing mechanism and a memory-control mechanism.

Batching does not change the set being computed. After each batch is processed, the running frontier is the nondominated union of all cutset products seen so far. Filtering a new batch against this frontier removes points already dominated by earlier batches, and the final mutual merge removes earlier points dominated by the new survivors. The result is therefore the same nondominated union over all active cutset nodes, but evaluated with bounded intermediate memory.

4 Computational Experiments

Our computational evaluation aims to answer the following key research questions:

RQ1 MIP vs. Serial DD: How does serial DD-based Pareto frontier enumeration compare with MIP baselines in runtime and scalability?

RQ2 Serial DD vs. Parallel DD on CPU: How much speedup does the parallel CPU DD implementation provide over serial DD?

Algorithm 3 GPU Subroutine: Batched Cutset Merge

Require: Cutset nodes $\mathcal{L}_c$; frontiers $\mathcal{Y}^\downarrow(v), \mathcal{Y}^\uparrow(v)$ and sizes $\kappa_v^\downarrow, \kappa_v^\uparrow$ for all $v \in \mathcal{L}_c$; candidate batch limit $B_{\max}$

- 1: $\mathcal{V}_c^* \leftarrow \{v \in \mathcal{L}_c \mid \kappa_v^\downarrow > 0 \text{ and } \kappa_v^\uparrow > 0\}$ $\triangleright$ Active nodes
- 2: $\text{score}(v) \leftarrow \kappa_v^\downarrow + \kappa_v^\uparrow$ for all $v \in \mathcal{V}_c^*$
- 3: Sort $\mathcal{V}_c^*$ in descending order of $\text{score}(v)$
- 4: $\mathcal{Y}^{\text{global}} \leftarrow \emptyset$ $\triangleright$ Running global frontier in GPU memory
- 5: Partition sorted $\mathcal{V}_c^*$ into batches $\mathcal{B}_1, \mathcal{B}_2, \dots$ s.t. $\sum_{v \in \mathcal{B}_k} \kappa_v^\downarrow \cdot \kappa_v^\uparrow \leq B_{\max}$
- 6: **for** each batch $\mathcal{B}$ **do**
- 7: $\beta \leftarrow \text{MATERIALIZEPRODUCTS}(\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B})$
- 8: **if** $\mathcal{Y}^{\text{global}} \neq \emptyset$ **then**
- 9: $\sigma \leftarrow \text{FILTERAGAINSTFRONTIER}(\beta, \mathcal{Y}^{\text{global}})$
- 10: $\beta \leftarrow \text{COMPACTSURVIVORS}(\beta, \sigma)$
- 11: **end if**
- 12: $\beta \leftarrow \text{SELFPRUNE}(\beta)$ $\triangleright$ Local 1D dominance check
- 13: $\mathcal{Y}^{\text{global}} \leftarrow \text{MERGEFRONTIERS}(\mathcal{Y}^{\text{global}}, \beta)$ $\triangleright$ Mutual pruning and concatenation
- 14: **end for**
- 15: **return** $\mathcal{Y}^{\text{global}}$

RQ3 Parallel DD on CPU vs. GPU: How does the GPU DD implementation compare with the parallel CPU DD variant in runtime and robustness?

4.1 Experimental Setup

We conducted all experiments on a compute cluster equipped with an Intel Xeon 6972P CPU running at 2.4 GHz and an NVIDIA H100 GPU. Across all experiments, the CPU memory limit was set to 16 GB and the GPU memory limit was set to 20 GB, with a one-hour time limit per instance. Our parallel PF enumeration algorithms were implemented on top of the C++ code of Bergman et al. (2022). Specifically, we used C++14 with OpenMP 4.5 for CPU parallelism and NVCC 12.6 for GPU acceleration.

We evaluate the proposed methods on MOKP, MOSPP, and MOTSP. The test instances are taken from Bergman et al. (2022). We report results for instance sizes on which their best method, namely bidirectional PF enumeration (reported as *Coup* or *Coup*⁺), required at least 5 seconds of total runtime. For each instance size, the test set contains 10 instances.

Bergman et al. (2022) showed that serial bidirectional PF enumeration (*Coup*/*Coup*⁺) outperforms the two prior criterion-space algorithms it was benchmarked against, Kirlik and Sayin (2014) and Ozlen, Burton, and MacRae (2014). Dächert, Fleuren, and Klamroth (2024) subsequently showed that their MIP-based defining point algorithm (DPA) outperforms both of these same methods, plus a third criterion-space baseline (Boland, Charkhgard, and Savelsbergh 2017). Neither paper compares directly against the other’s paradigm, so we benchmark our DD-based methods directly against DPA, rather than against the methods it has already superseded, to test whether DD-based enumeration remains competitive with the strongest available MIP-based approach. We denote the serial bidirectional PF enumeration method of Bergman et al. (2022) by *Coup* or

Coup⁺. The “+” suffix indicates that state-dominance-based label pruning is used in addition to bidirectional PF enumeration. To ensure a fair comparison, we use the implementation provided by Bergman et al. (2022), rather than our modified implementation, which can also perform serial PF enumeration. We refer to the two variants of DPA as DPA-A and DPA-TS.

The CPU bidirectional PF enumeration variants are reported as *Coup*-T (or *Coup*⁺-T), where T specifies the number of parallel threads. The GPU bidirectional PF enumeration variants are reported as G-*Coup* and G-*Coup*⁺. We report the enumeration wall-clock time in seconds (“Time (s)”), the speed-up w.r.t. *Coup* or *Coup*⁺, and the tuple “(M, T)”, which gives the number of instances that failed due to out-of-memory errors (M) and time-out limits (T).

4.2 Results

RQ1 (MIP vs. Serial DD). Table 1 shows that serial DD enumeration is substantially more robust than the DPA baselines on MOKP and MOTSP. On MOKP, DPA solves only two instances for $(N, K) = (40, 6)$ and none of the remaining reported settings, whereas *Coup* solves all instances for $N = 40$ and remains competitive on the larger $N = 70$ settings. On MOTSP, both DPA variants time out on all reported settings, while *Coup* solves all instances for $K = 4, 5$, nine instances for $K = 6$, and one instance for $K = 7$. The MOSPP results are more mixed: DPA-A is faster than *Coup* for $(150, 3)$ and $(200, 3)$, and solves all instances where *Coup* encounters memory failures for $N = 200$; however, DPA fails on the high-objective $(150, 7)$ setting where *Coup* solves all instances. This crossover follows from how each method’s cost scales. DPA solves for each nondominated point individually, so its cost scales with frontier cardinality, and MOSPP’s frontier stays small at low K , exactly where DPA-A is fastest. DD-based enumeration instead scales with the size of the underlying state space and the intermediate labels it carries through that space. MOSPP’s state space is large even at low K , so it generates far more intermediate labels than its small final frontier would suggest, which is what makes low- K MOSPP comparatively hard for DD-based enumeration. As K grows, this reverses, since the frontier itself grows large enough that solving one MIP per point becomes DPA’s bottleneck, while DD-based enumeration continues to benefit from sharing structure across the state space. The two families thus fail in complementary regimes, trading off which quantity, state-space size or frontier cardinality, becomes the bottleneck as the number of objectives grows. This confirms that DD-based enumeration remains competitive with the strongest available MIP baseline, not merely with the older methods it has already superseded.

RQ2 (Serial DD vs. Parallel DD on CPU). The CPU variants provide consistent but bounded speedups over serial DD enumeration. On MOKP, the best CPU speedups range from $1.80\times$ to $3.21\times$, with weaker scaling on the larger $N = 70$ cases. On MOSPP, CPU speedups range from about $1.51\times$ on the easiest setting to $4.12\times$ for $(200, 4)$.

MOTSP benefits most from CPU parallelism: `Coup-16` reaches $7.83\times$ for $K = 4$ and $5.05\times$ for $K = 5$, while still improving over serial enumeration for $K = 6, 7$. The CPU variants also improve robustness in several harder settings, solving more instances than serial `Coup` on large MOKP and MOTSP cases.

RQ3 (Parallel DD on CPU vs. GPU). The GPU variants are often fastest when the intermediate frontiers fit in GPU memory, but they are less robust than the CPU variants. On MOKP, GPU methods dominate the small $N = 40$ settings, with `G-Coup` reaching $5.51\times$ and $9.71\times$ speedups; for $N = 70$, `G-TD` is fastest on solved instances but incurs GPU out-of-memory failures. On MOSPP, GPU performance is strong for $N = 150$, including a $20.88\times$ speedup for `G-TD` at $(150, 7)$, but both GPU variants fail due to memory on the reported $N = 200$ settings. On MOTSP, GPU methods are competitive with or faster than the CPU variants, especially for $K = 5, 6$, but memory failures appear for `G-TD` at higher K and for `G-Coup` at $K = 7$. Overall, the GPU gives the largest speedups, while the parallel CPU variants are more reliable under the imposed memory limits. This gap in robustness follows partly from how the two implementations are structured: the GPU bounds each layer’s candidate materialization to a fixed-size batch of nodes, applying dominance filtering between batches so that many candidates are discarded before the rest of the layer is expanded, but it also keeps separate flattened device-side representations for top-down and bottom-up traversal, which doubles its baseline GPU memory footprint before enumeration even starts. The CPU’s node-owned frontier degrades more gracefully, since a large node increases one worker’s execution time instead of exhausting a memory pool shared by all workers. Full tables for all reported instance sizes are provided in Appendix C.

5 Related Work

We position our contribution against two strands of prior work. The first, exact algorithms for multiobjective discrete optimization, supplies the enumeration model on which DD-based Pareto frontier construction is built; the second, parallel computing for discrete optimization, addresses how the computational kernels of exact solvers are accelerated on multi-core and many-core hardware. We review each in turn before situating our contribution relative to both.

5.1 Exact Multiobjective Discrete Optimization

Exact algorithms for multiobjective discrete optimization are commonly divided into criterion-space, decision-space, and dynamic-programming or state-space approaches; see Ehrgott, Gandibleux, and Przybylski (2016); Halffmann et al. (2022) for surveys. Criterion-space methods enumerate nondominated points by repeatedly solving scalarized or constrained single-objective subproblems while partitioning the unexplored objective space. Representative examples include recursive and box-decomposition schemes (Özlen and Azizoğlu 2009; Ozlen, Burton, and MacRae 2014; Boland, Charkhgard, and Savelsbergh 2015), parameter-space search

(Kirlik and Sayin 2014), and recent objective-space algorithms for multiobjective integer programming (Dächert, Fleuren, and Klamroth 2024; Tamby and Vanderpooten 2021). Decision-space methods instead generalize branch-and-bound or branch-and-cut to vector-valued bounds and dominance-based fathoming.

Dynamic-programming and state-space methods propagate sets of nondominated labels through a recursive model. Decision diagrams belong to this family. Bergman and Cire (2016) introduced multiobjective optimization by DDs, and Bergman et al. (2022) developed network models and reduction operations that make exact Pareto frontier enumeration competitive when a compact DD representation is available. Our work builds on this DD-based state-space approach. We do not change the exact enumeration model; instead, we parallelize its dominant operations: label generation, dominance filtering, cutset-product materialization, and frontier merging.

5.2 Parallel Computing for Discrete Optimization

Parallel computing has long been used to accelerate exact discrete optimization (Kindervater and Lenstra 1986). Much of the classical work focuses on branch-and-bound, where parallelism comes from distributing search-tree nodes across processors (Gendron and Crainic 1994; Ralphs, Ladányi, and Saltzman 2003; Linderoth and Wright 2003). Such methods can scale well when the tree is effectively partitioned, but they also exhibit search-order effects such as the parallel branch-and-bound anomalies studied by Lai and Sahni (1984). Parallelism has also been studied in DD-based optimization: Bergman et al. (2014) proposed a parallel DD-based branch-and-bound framework that distributes DD node exploration across processors.

More recent work has explored accelerator hardware. GPU implementations have been proposed for parts of simplex and linear-programming solvers (Lalami, Boyer, and El-Baz 2011; Ploskas and Samaras 2016; Montoisson et al. 2025), and GPU-native first-order methods have shown strong performance on large-scale linear programs (Apple-gate et al. 2021; Lu and Yang 2025; Lu et al. 2023; Lu, Peng, and Yang 2025). For mixed-integer programming, GPU acceleration is harder because search trees are irregular; existing approaches typically keep the tree control on the CPU and offload selected kernels or heuristics to the GPU (Boyer, El Baz, and Salazar-Aguilar 2017; Çördük et al. 2025). Closest to our setting is the GPU-accelerated state expansion framework for anytime exact DD search of Tardivo, Michel, and van Hoeve (2026). In contrast, we study exact Pareto frontier enumeration on DDs, where the main parallel kernels are not tree-node relaxations but label generation, pairwise dominance filtering, and nondominated-set reduction.

6 Conclusion and Future Work

This paper parallelized exact bidirectional PF enumeration on DDs, exploiting parallelism at the level of nodes within a layer, labels within a node, and products across the cutset. The CPU implementation delivers consistent speedups over serial enumeration, up to $7.83\times$ on MOTSP, while the GPU

Table 1: Pareto Enumeration Runtime Comparison Across Problems and Enumeration Methods

Problem	N	K	Method	Time (s)	Speed-up	$S^{(M,T)}$	N	K	Method	Time (s)	Speed-up	$S^{(M,T)}$
MOKP	40	6	DPA-TS	1371	$0.01 \times$	$2^{(0,8)}$	40	7	DPA-TS	–	–	$0^{(0,10)}$
			DPA-A	1439	$0.01 \times$	$2^{(0,8)}$			DPA-A	–	–	$0^{(0,10)}$
			Coup	10	$1.00 \times$	10			Coup	33	$1.00 \times$	10
			Coup-4	6	$1.87 \times$	10			Coup-4	16	$2.12 \times$	10
			Coup-8	5	$2.15 \times$	10			Coup-8	13	$2.63 \times$	10
			Coup-16	4	$2.60 \times$	10			Coup-16	11	$3.21 \times$	10
			G-TD	3	$4.31 \times$	10			G-TD	5	$7.12 \times$	10
			G-Coup	2	$5.51 \times$	10			G-Coup	4	$9.71 \times$	10
	70	6	DPA-TS	–	–	$0^{(0,10)}$	70	7	DPA-TS	–	–	$0^{(0,10)}$
			DPA-A	–	–	$0^{(0,10)}$			DPA-A	–	–	$0^{(0,10)}$
			Coup	1549	$1.00 \times$	$5^{(0,5)}$			Coup	1650	$1.00 \times$	$3^{(0,7)}$
			Coup-4	1326	$1.17 \times$	$7^{(0,3)}$			Coup-4	1391	$1.19 \times$	$4^{(0,6)}$
			Coup-8	1008	$1.54 \times$	$7^{(0,3)}$			Coup-8	1757	$0.94 \times$	$6^{(0,4)}$
			Coup-16	862	$1.80 \times$	$7^{(0,3)}$			Coup-16	1708	$0.97 \times$	$7^{(0,3)}$
			G-TD	335	$4.63 \times$	$6^{(4,0)}$			G-TD	372	$4.44 \times$	$3^{(7,0)}$
			G-Coup	1010	$1.53 \times$	10			G-Coup	907	$1.82 \times$	$6^{(3,1)}$
MOSPP	150	3	DPA-TS	11	$1.12 \times$	10	150	7	DPA-TS	–	–	$0^{(0,10)}$
			DPA-A	8	$1.59 \times$	10			DPA-A	–	–	$0^{(0,10)}$
			Coup	12	$1.00 \times$	10			Coup	184	$1.00 \times$	10
			Coup-4	8	$1.50 \times$	10			Coup-4	84	$2.20 \times$	10
			Coup-8	8	$1.50 \times$	10			Coup-8	71	$2.61 \times$	10
			Coup-16	8	$1.51 \times$	10			Coup-16	96	$1.91 \times$	10
			G-TD	7	$1.80 \times$	10			G-TD	9	$20.88 \times$	10
			G-Coup	5	$2.44 \times$	$9^{(1,0)}$			G-Coup	17	$10.98 \times$	10
	200	3	DPA-TS	32	$5.15 \times$	10	200	4	DPA-TS	913	$0.20 \times$	10
			DPA-A	22	$7.53 \times$	10			DPA-A	793	$0.23 \times$	10
			Coup	162	$1.00 \times$	$1^{(9,0)}$			Coup	186	$1.00 \times$	$1^{(9,0)}$
			Coup-4	93	$1.75 \times$	$1^{(9,0)}$			Coup-4	80	$2.34 \times$	$1^{(9,0)}$
			Coup-8	87	$1.86 \times$	$1^{(9,0)}$			Coup-8	63	$2.97 \times$	$1^{(9,0)}$
			Coup-16	72	$2.27 \times$	$1^{(8,1)}$			Coup-16	45	$4.12 \times$	$1^{(9,0)}$
			G-TD	–	–	$0^{(10,0)}$			G-TD	–	–	$0^{(10,0)}$
			G-Coup	–	–	$0^{(10,0)}$			G-Coup	–	–	$0^{(10,0)}$
MOTSP	15	4	DPA-TS	–	–	$0^{(0,10)}$	15	5	DPA-TS	–	–	$0^{(0,10)}$
			DPA-A	–	–	$0^{(0,10)}$			DPA-A	–	–	$0^{(0,10)}$
			Coup	23	$1.00 \times$	10			Coup	335	$1.00 \times$	10
			Coup-4	7	$3.47 \times$	10			Coup-4	115	$2.92 \times$	10
			Coup-8	5	$5.06 \times$	10			Coup-8	82	$4.11 \times$	10
			Coup-16	3	$7.83 \times$	10			Coup-16	67	$5.05 \times$	10
			G-TD	3	$7.50 \times$	10			G-TD	36	$9.44 \times$	$9^{(1,0)}$
			G-Coup	3	$8.03 \times$	10			G-Coup	48	$7.04 \times$	10
	15	6	DPA-TS	–	–	$0^{(0,10)}$	15	7	DPA-TS	–	–	$0^{(0,10)}$
			DPA-A	–	–	$0^{(0,10)}$			DPA-A	–	–	$0^{(0,10)}$
			Coup	1623	$1.00 \times$	$9^{(0,1)}$			Coup	3544	$1.00 \times$	$1^{(0,9)}$
			Coup-4	870	$1.87 \times$	10			Coup-4	2508	$1.41 \times$	$3^{(0,7)}$
			Coup-8	711	$2.28 \times$	10			Coup-8	2123	$1.67 \times$	$3^{(0,7)}$
			Coup-16	641	$2.53 \times$	10			Coup-16	1930	$1.84 \times$	$3^{(0,7)}$
			G-TD	–	–	$0^{(10,0)}$			G-TD	–	–	$0^{(10,0)}$
			G-Coup	369	$4.40 \times$	10			G-Coup	1828	$1.94 \times$	$6^{(3,1)}$

implementation pushes decomposition below the level of a single node to reach up to $20.88\times$ on MOSPP but is more memory-bound as a result. Relative to the MIP baseline, DD-based enumeration is markedly more robust on MOKP and MOTSP, though the MIP baseline remains competitive on large, low-objective MOSPP instances where DD-based frontiers grow large. Both gaps trace back to the same bottleneck: intermediate frontier size relative to available memory.

Since that bottleneck is shared, we see two immediate directions and one longer-term one. On the implementation side, our GPU implementation keeps separate flattened top-down and bottom-up representations that double its baseline device-memory footprint; unifying them into one shared representation, together with a scheduler that routes each layer expansion or cutset batch to whichever of the CPU or GPU its memory footprint favors, would let a single run keep the GPU's speed without its memory failures. Since the MIP baseline wins precisely where DD-based frontiers overrun memory, a solver that monitors frontier growth and falls back to a MIP-style box search before memory is exhausted could combine the objective-scalability of DD-based enumeration with the memory-independence of the MIP baseline. A related, longer-term limitation is that our approach assumes a full exact DD already exists: the out-of-memory failures we report ultimately stem from holding this complete structure, and its complete node frontiers, in memory at once. Embedding DD-based PF enumeration in a branch-and-bound scheme over restricted and relaxed diagrams (Bergman et al. 2016), which yield primal and dual bound sets without ever building the full exact diagram, would address this pressure directly, and the label-propagation and dominance-filtering kernels we parallelize here would carry over unchanged to speed up bound recovery in that setting.

This asymmetry means neither implementation dominates the other, and the choice between them is information available at run time rather than something fixed in advance. A natural extension is a scheduler that routes each layer expansion or cutset batch to whichever device its size and memory footprint favor, falling back to the CPU when a node frontier would overflow GPU memory and to the GPU when it would not. More broadly, the batching already used in the cutset merge, splitting products into memory-bounded chunks against a running frontier, applies within a single layer; extending it so layer expansion also streams through fixed-size candidate buffers, instead of materializing an entire layer at once, would let the GPU implementation trade some of its speed for the graceful degradation the CPU implementation already exhibits.

References

- Applegate, D.; Diaz, M.; Hinder, O.; Lu, H.; Lubin, M.; O'Donoghue, B.; and Schudy, W. 2021. Practical large-scale linear programming using primal-dual hybrid gradient. In *Advances in Neural Information Processing Systems*, volume 34, 20243–20257.
- Bergman, D.; Bodur, M.; Cardonha, C.; and Cire, A. A. 2022. Network models for multiobjective discrete optimization. *INFORMS Journal on Computing*, 34(2): 990–1005.
- Bergman, D.; and Cire, A. A. 2016. Multiobjective optimization by decision diagrams. In *International Conference on Principles and Practice of Constraint Programming*, 86–95. Springer.
- Bergman, D.; Cire, A. A.; Sabharwal, A.; Samulowitz, H.; and Van Hoeve, W.-J. 2014. Parallel combinatorial optimization with decision diagrams. In *International Conference on AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CPAIOR)*, 351–367. Springer.
- Bergman, D.; Cire, A. A.; Van Hoeve, W.-J.; and Hooker, J. 2016. *Decision diagrams for optimization*, volume 1. Springer.
- Boland, N.; Charkhgard, H.; and Savelsbergh, M. 2015. A criterion space search algorithm for biobjective integer programming: The balanced box method. *INFORMS Journal on Computing*, 27(4): 735–754.
- Boland, N.; Charkhgard, H.; and Savelsbergh, M. 2017. A new method for optimizing a linear function over the efficient set of a multiobjective integer program. *European Journal of Operational Research*, 260(3): 904–919.
- Boyer, V.; El Baz, D.; and Salazar-Aguilar, M. A. 2017. GPU computing applied to linear and mixed-integer programming.
- Çördük, A.; Sielski, P.; Boucher, A.; and Aatish, K. 2025. GPU-Accelerated Primal Heuristics for Mixed Integer Programming. *arXiv preprint arXiv:2510.20499*.
- Dächert, K.; Fleuren, T.; and Klamroth, K. 2024. A simple, efficient and versatile objective space algorithm for multi-objective integer programming. *Mathematical Methods of Operations Research*, 100(1): 351–384.
- Deb, K.; Agrawal, S.; Pratap, A.; and Meyarivan, T. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.*, 6: 182–197.
- Ehrgott, M. 2005. *Multicriteria optimization*. Springer.
- Ehrgott, M.; Gandibleux, X.; and Przybylski, A. 2016. Exact methods for multi-objective combinatorial optimisation. *Multiple criteria decision analysis: State of the art surveys*, 817–850.
- Gendron, B.; and Crainic, T. G. 1994. Parallel branch-and-branch algorithms: Survey and synthesis. *Operations research*, 42(6): 1042–1066.
- Halffmann, P.; Schäfer, L. E.; Dächert, K.; Klamroth, K.; and Ruzika, S. 2022. Exact algorithms for multiobjective linear optimization problems with integer variables: A state of the art survey. *Journal of Multi-Criteria Decision Analysis*, 29(5-6): 341–363.
- Kindervater, G. A.; and Lenstra, J. K. 1986. An introduction to parallelism in combinatorial optimization. *Discrete Applied Mathematics*, 14(2): 135–156.
- Kirlik, G.; and Sayın, S. 2014. A new algorithm for generating all nondominated solutions of multiobjective discrete optimization problems. *European Journal of Operational Research*, 232(3): 479–488.

- Kovacs, A. A.; Parragh, S. N.; and Hartl, R. F. 2015. The multi-objective generalized consistent vehicle routing problem. *European Journal of Operational Research*, 247(2): 441–458.
- Lai, T.-H.; and Sahni, S. 1984. Anomalies in parallel branch-and-bound algorithms. *Communications of the ACM*, 27(6): 594–602.
- Lalami, M. E.; Boyer, V.; and El-Baz, D. 2011. Efficient implementation of the simplex method on a CPU-GPU system. In *2011 IEEE International Symposium on Parallel and Distributed Processing Workshops and Phd Forum*, 1999–2006. IEEE.
- Linderoth, J.; and Wright, S. 2003. Decomposition algorithms for stochastic programming on a computational grid. *Computational Optimization and Applications*, 24(2): 207–250.
- Lindholm, E.; Nickolls, J.; Oberman, S.; and Montrym, J. 2008. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE micro*, 28(2): 39–55.
- Lu, H.; Peng, Z.; and Yang, J. 2025. cuPDLPx: A further enhanced GPU-based first-order solver for linear programming. *arXiv preprint arXiv:2507.14051*.
- Lu, H.; and Yang, J. 2025. cuPDLP.jl: A GPU implementation of restarted primal-dual hybrid gradient for linear programming in Julia. *Operations Research*, 73(6): 3440–3452.
- Lu, H.; Yang, J.; Hu, H.; Huangfu, Q.; Liu, J.; Liu, T.; Ye, Y.; Zhang, C.; and Ge, D. 2023. cuPDLP-C: A strengthened implementation of cuPDLP for linear programming by C language. *arXiv preprint arXiv:2312.14832*.
- Miettinen, K. 1999. *Nonlinear multiobjective optimization*, volume 12. Springer Science & Business Media.
- Montoisson, A.; Pacaud, F.; Shin, S.; and Anitescu, M. 2025. GPU Implementation of Second-Order Linear and Nonlinear Programming Solvers. *arXiv preprint arXiv:2508.16094*. Describes the MadIPM solver.
- Nickolls, J.; Buck, I.; Garland, M.; and Skadron, K. 2008. Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for? *Queue*, 6(2): 40–53.
- Nickolls, J.; and Dally, W. J. 2010. The GPU computing era. *IEEE micro*, 30(2): 56–69.
- Özlen, M.; and Azizoğlu, M. 2009. Multi-objective integer programming: A general approach for generating all non-dominated solutions. *European Journal of Operational Research*, 199(1): 25–35.
- Ozlen, M.; Burton, B. A.; and MacRae, C. A. 2014. Multi-objective integer programming: An improved recursive algorithm. *Journal of Optimization Theory and Applications*, 160(2): 470–482.
- Ploskas, N.; and Samaras, N. 2016. GPU accelerated pivoting rules for the simplex algorithm. *Journal of Systems and Software*, 117: 513–524.
- Ralphs, T. K.; Ladányi, L.; and Saltzman, M. J. 2003. Parallel branch, cut, and price for large-scale discrete optimization. *Mathematical Programming*, 98(1): 253–280.
- Tamby, S.; and Vanderpooten, D. 2021. Enumeration of the nondominated set of multiobjective discrete optimization problems. *INFORMS Journal on Computing*, 33(1): 72–85.
- Tardivo, F.; Michel, L.; and van Hoeve, W.-J. 2026. GPU-Accelerated State Expansion for Anytime Exact Decision-Diagram Search. In *Proceedings of the 23rd International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2026)*, Lecture Notes in Computer Science. Springer. To appear.
- Vianna, D.; and Arroyo, J. 2004. A GRASP algorithm for the multi-objective knapsack problem. In *XXIV International Conference of the Chilean Computer Science Society*, 69–75.

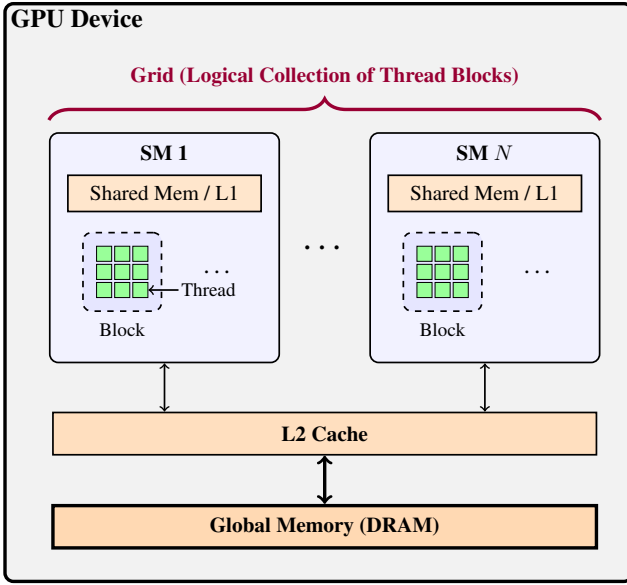


Figure 1: Schematic representation of a GPU architecture.

A Graphics Processing Unit

GPUs are throughput-oriented processors that achieve high performance by executing many lightweight threads in parallel. In contrast, CPUs comprise a small number of complex cores optimized for low-latency execution and sophisticated control flow. This architectural difference makes GPUs particularly effective for data-parallel workloads in which the same operation is applied independently across many inputs (Lindholm et al. 2008). Figure 1 illustrates the GPU execution and memory hierarchy. Computation is launched as a *kernel*, which executes as a large collection of *threads*. Threads are grouped into *thread blocks*, and all blocks in a kernel launch form a *grid* (Nickolls et al. 2008). The GPU schedules thread blocks onto *Streaming Multiprocessors* (SMs), where each SM can host multiple blocks concurrently and executes threads in a *Single Instruction Multiple Thread* (SIMT) manner, where threads follow the same instruction stream while operating on different data.

The memory system is also hierarchical. Threads within an SM have access to fast on-chip storage, including registers and shared memory, which enables efficient intra-block cooperation and synchronization. SMs connect to a larger, device-wide cache (L2) and ultimately to high-capacity but high-latency global memory (DRAM). Effective GPU implementations therefore aim to expose ample parallelism while limiting expensive global-memory traffic (Nickolls and Dally 2010). This execution model is well suited for workloads dominated by large numbers of independent arithmetic operations and comparisons. In particular, pairwise dominance checks required during PF enumeration in Algorithm 1 are natural candidates for GPU acceleration.

B Implementation Details

This appendix gives pseudocode for implementation details referenced in the main text but omitted there for space. The CPU detail is the pairwise reduction used to merge private cutset frontiers. The GPU details are the lower-level kernels used by Algorithm 2.

B.1 CPU Implementation

The CPU implementation uses the tree reduction in Algorithm 4 to merge private cutset frontiers produced by different workers. This is the reduction referred to in Section 3.2.

Algorithm 4 CPU Tree Reduction for Cutset Frontiers

Require: Private cutset frontiers $\mathcal{P} = [\mathcal{P}_1, \dots, \mathcal{P}_q]$

```

1: procedure TREEREDUCE( $\mathcal{P}$ )
2:   while  $|\mathcal{P}| > 1$  do
3:      $\mathcal{Q} \leftarrow []$ 
4:     for all  $i = 1, 3, 5, \dots, 2\lfloor |\mathcal{P}|/2 \rfloor - 1$  in parallel
5:       do
6:          $\mathcal{Q}_{(i+1)/2} \leftarrow \text{FILTERDOMINATED}(\mathcal{P}_i \cup \mathcal{P}_{i+1})$ 
7:       end for
8:       if  $|\mathcal{P}|$  is odd then
9:          $\mathcal{Q}_{\lfloor |\mathcal{P}|/2 \rfloor + 1} \leftarrow \mathcal{P}_{|\mathcal{P}|}$ 
10:      end if
11:       $\mathcal{P} \leftarrow \mathcal{Q}$ 
12:    end while
13:  return  $\mathcal{P}_1$ 
14: end procedure

```

B.2 GPU Implementation

The GPU implementation expands each selected layer through the four-kernel pipeline described in Section 3.3. Algorithm 5 computes per-arc candidate counts and buffer offsets. Algorithm 6 then materializes shifted candidate labels in the assigned buffer segments. Algorithm 9 performs load-balanced dominance filtering, and Algorithm 7 compacts the surviving labels into the next node-frontier representation. Finally, Algorithm 8 materializes the batched cutset products used by Algorithm 3.

Algorithm 6 uses the offsets from Algorithm 5 to write every generated candidate into a disjoint location of the flat buffer. Algorithm 9 assigns more thread blocks to nodes with larger candidate segments, so the dominance-filtering work is balanced at the level of node-frontier size. Algorithm 7 converts the binary survivor flags produced by the dominance kernel into compacted node-frontier storage. Algorithm 8 is used during the batched cutset merge. It materializes the Cartesian products for a batch of active cutset nodes before filtering and merging them into the running global frontier.

C Additional Experimental Results

This appendix reports the full set of computational results summarized in Table 1. The full MOKP, MOSPP, and MOTSP results are given in Tables 2 to 4, respectively.

Algorithm 5 Compute Candidate Offsets

Require: Frontier $\mathcal{Y}$; source-node array $b[]$; incoming-arc offsets $\phi^a[]$

- 1: **procedure** COMPUTEARCOFFSETS($\mathcal{Y}, b, \phi^a$)
- 2: $M \leftarrow |\mathcal{A}_{\mathcal{L}_j}|$ $\triangleright$ Number of arcs in layer j
- 3: **for all** arcs $a = 0$ **to** $M - 1$ **in parallel on GPU do**
 $\triangleright$ One thread per arc
- 4: $\kappa_a \leftarrow |\mathcal{Y}(b(a))|$
- 5: **end for**
- 6: $\phi_0^c \leftarrow 0$
- 7: **for** $a = 1$ **to** M **in parallel on GPU do**
- 8: $\phi_a^c \leftarrow \phi_{a-1}^c + \kappa_{a-1}$
- 9: **end for**
- 10: **for all** nodes $v \in \mathcal{L}_{j+1}$ **in parallel on GPU do**
- 11: $a_{\text{first}} \leftarrow \phi_v^a$
- 12: $a_{\text{last}} \leftarrow \phi_{v+1}^a$
- 13: $\Gamma_v \leftarrow \phi_{a_{\text{last}}}^c - \phi_{a_{\text{first}}}^c$
- 14: **end for**
- 15: **return** κ, ϕ^c, Γ
- 16: **end procedure**

Algorithm 6 Materialize Layer Candidates

Require: Frontier $\mathcal{Y}$; source array $b[]$; arc weights $\mathbf{v}(\cdot)$; off-sets $\phi^c[]$; counts $\kappa[]$

- 1: **procedure** EXPANDCANDIDATES($\mathcal{Y}, b, \mathbf{v}, \phi^c, \kappa$)
- 2: Allocate flat buffer β of size $\sum_a \kappa_a$
- 3: **for all** arcs a **in parallel on GPU do**
- 4: // One warp (32 threads) per arc
- 5: **for all** $k = 0, 1, \dots, \kappa_a - 1$ **distributed across warp threads do**
- 6: $\beta[\phi_a^c + k] \leftarrow \mathcal{Y}(b(a))[k] + \mathbf{v}(a)$
- 7: **end for**
- 8: **end for**
- 9: **return** β
- 10: **end procedure**

Algorithm 7 Compact Node Frontiers

Require: Candidate buffer β of size S ; binary survivor flags $\sigma[]$

- 1: **procedure** COMPACTSURVIVORS(β, σ)
- 2: $\rho_0 \leftarrow 0$
- 3: **for** $i = 1$ **to** S **do** $\triangleright$ Parallel prefix sum on GPU
- 4: $\rho_i \leftarrow \rho_{i-1} + \sigma_{i-1}$
- 5: **end for**
- 6: **for all** candidates $i = 0$ **to** $S - 1$ **in parallel on GPU do**
 $\triangleright$ One thread per candidate
- 7: **if** $\sigma_i = 1$ **then**
- 8: $\mathcal{Y}(v)[\rho_i] \leftarrow \beta[i]$ $\triangleright v$: destination node of i
- 9: **end if**
- 10: **end for**
- 11: **return** $\mathcal{Y}$
- 12: **end procedure**

The complete MOKP runtime comparison is shown in Table 2. The complete MOSPP runtime comparison is shown

Algorithm 8 Materialize Cutset Products

Require: Frontiers $\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow$; node offsets $\phi^\downarrow, \phi^\uparrow$ and sizes $\kappa^\downarrow, \kappa^\uparrow$ for nodes in batch $\mathcal{B}$; product offsets ϕ^p

- 1: **procedure** MATERIALIZEPRODUCTS($\mathcal{Y}^\downarrow, \mathcal{Y}^\uparrow, \mathcal{B}$)
- 2: Allocate flat candidate buffer β of size $\sum_{v \in \mathcal{B}} \kappa_v^\downarrow \cdot \kappa_v^\uparrow$
- 3: **for all** nodes $v \in \mathcal{B}$ **in parallel on GPU do** $\triangleright$ One thread block per node
- 4: $T \leftarrow \kappa_v^\downarrow \cdot \kappa_v^\uparrow$ $\triangleright$ Total candidates for node v
- 5: **for all** candidate indices $p = 0, \dots, T - 1$ **in parallel do**
- 6: $i \leftarrow \lfloor p / \kappa_v^\uparrow \rfloor$; $j \leftarrow p \pmod{\kappa_v^\uparrow}$
- 7: $y^\downarrow \leftarrow \mathcal{Y}^\downarrow(\phi_v^\downarrow + i)$; $y^\uparrow \leftarrow \mathcal{Y}^\uparrow(\phi_v^\uparrow + j)$
- 8: $\beta[\phi_v^p + p] \leftarrow y^\downarrow + y^\uparrow$
- 9: **end for**
- 10: **end for**
- 11: **return** β
- 12: **end procedure**

in Table 3.

The complete MOTSP runtime comparison is shown in Table 4.

Algorithm 9 Load-Balanced GPU Dominance Filter

Require: Flat candidate buffer β ; per-node candidate counts $\Gamma[\cdot]$; block size B

```
1: procedure FILTERDOMINATED( $\beta, \Gamma$ )
2:    $K[v] \leftarrow \lceil \Gamma[v]/B \rceil$  for all  $v$ 
3:    $\phi_0^k \leftarrow 0$ 
4:   for  $v = 1$  to  $|\mathcal{L}_{j+1}|$  in parallel on GPU do
5:      $\phi_v^k \leftarrow \phi_{v-1}^k + K[v - 1]$ 
6:   end for
7:    $\sigma_i \leftarrow 1$  for all candidates  $i$ 
8:   for all thread blocks  $q = 0, \dots, \sum_v K[v] - 1$  in parallel on GPU do
9:     Binary search  $\phi^k$  to find node  $v$  s.t.  $\phi_v^k \leq q < \phi_{v+1}^k$ 
10:     $\tau \leftarrow q - \phi_v^k$   $\triangleright$  Tile index within node  $v$ 
11:     $[b, e) \leftarrow$  candidate range of node  $v$  in  $\beta$ 
12:     $i \leftarrow b + \tau \cdot B +$  thread index within block
13:    Load  $\mathbf{y}_i \leftarrow \beta[i]$  into per-thread register (if  $i < e$ )  $\triangleright$  Reused across tiles
14:     $\text{dominated} \leftarrow \text{false}$ 
15:    for tile start  $s = b$  to  $e-1$ , step  $B$  do
16:      Barrier: wait for all threads in block
17:      Each thread  $k$  loads  $\beta[s+k]$  into shared memory slot  $k$  (if  $s+k < e$ )
18:      Barrier: ensure tile is fully visible
19:      if  $i < e$  and not  $\text{dominated}$  then
20:        for  $j = 0$  to  $\min(B, e-s)-1$ , skip  $j = i-b-\tau B$  do
21:          if  $\text{shared}[j] \prec \mathbf{y}_i$  then
22:             $\text{dominated} \leftarrow \text{true}; \text{break}$ 
23:          end if
24:        end for  $\triangleright$  Early exit within tile
25:      end if
26:      if  $\text{dominated}$  then break
27:      end if  $\triangleright$  Early exit across tiles
28:      end for
29:      if  $i < e$  and  $\text{dominated}$  then  $\sigma_i \leftarrow 0$ 
30:      end if
31:    end for
32:  return  $\sigma$ 
33: end procedure
```

Table 2: MOKP Pareto Enumeration Runtime Comparison Across Enumeration Methods

N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$	N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$
40	6	DPA-TS	1371	$0.01 \times$	$2^{(0,8)}$	40	7	DPA-TS	—	—	$0^{(0,10)}$
		DPA-A	1439	$0.01 \times$	$2^{(0,8)}$			DPA-A	—	—	$0^{(0,10)}$
		Coup	10	$1.00 \times$	10			Coup	33	$1.00 \times$	10
		Coup-4	6	$1.87 \times$	10			Coup-4	16	$2.12 \times$	10
		Coup-8	5	$2.15 \times$	10			Coup-8	13	$2.63 \times$	10
		Coup-16	4	$2.60 \times$	10			Coup-16	11	$3.21 \times$	10
		G-TD	3	$4.31 \times$	10			G-TD	5	$7.12 \times$	10
		G-Coup	2	$5.51 \times$	10			G-Coup	4	$9.71 \times$	10
50	5	DPA-TS	1752	$0.01 \times$	$5^{(0,5)}$	50	6	DPA-TS	—	—	$0^{(0,10)}$
		DPA-A	1625	$0.01 \times$	$5^{(0,5)}$			DPA-A	—	—	$0^{(0,10)}$
		Coup	19	$1.00 \times$	10			Coup	184	$1.00 \times$	10
		Coup-4	9	$2.06 \times$	10			Coup-4	82	$2.26 \times$	10
		Coup-8	8	$2.42 \times$	10			Coup-8	63	$2.94 \times$	10
		Coup-16	7	$2.93 \times$	10			Coup-16	52	$3.59 \times$	10
		G-TD	3	$6.49 \times$	10			G-TD	45	$4.09 \times$	10
		G-Coup	4	$5.89 \times$	10			G-Coup	26	$7.12 \times$	10
50	7	DPA-TS	—	—	$0^{(0,10)}$	60	4	DPA-TS	1102	$0.01 \times$	$9^{(0,1)}$
		DPA-A	—	—	$0^{(0,10)}$			DPA-A	925	$0.02 \times$	$9^{(0,1)}$
		Coup	499	$1.00 \times$	10			Coup	16	$1.00 \times$	10
		Coup-4	229	$2.18 \times$	10			Coup-4	8	$2.04 \times$	10
		Coup-8	180	$2.78 \times$	10			Coup-8	7	$2.36 \times$	10
		Coup-16	150	$3.34 \times$	10			Coup-16	6	$3.01 \times$	10
		G-TD	81	$6.20 \times$	10			G-TD	3	$7.74 \times$	10
		G-Coup	82	$6.08 \times$	10			G-Coup	3	$6.98 \times$	10
60	5	DPA-TS	—	—	$0^{(0,10)}$	60	6	DPA-TS	—	—	$0^{(0,10)}$
		DPA-A	—	—	$0^{(0,10)}$			DPA-A	—	—	$0^{(0,10)}$
		Coup	190	$1.00 \times$	10			Coup	849	$1.00 \times$	$8^{(0,2)}$
		Coup-4	84	$2.27 \times$	10			Coup-4	678	$1.25 \times$	10
		Coup-8	67	$2.83 \times$	10			Coup-8	424	$2.00 \times$	10
		Coup-16	57	$3.34 \times$	10			Coup-16	341	$2.49 \times$	10
		G-TD	30	$6.31 \times$	10			G-TD	201	$4.23 \times$	10
		G-Coup	30	$6.31 \times$	10			G-Coup	156	$5.45 \times$	10
60	7	DPA-TS	—	—	$0^{(0,10)}$	70	3	DPA-TS	87	$0.05 \times$	10
		DPA-A	—	—	$0^{(0,10)}$			DPA-A	62	$0.07 \times$	10
		Coup	1244	$1.00 \times$	$6^{(0,4)}$			Coup	5	$1.00 \times$	10
		Coup-4	969	$1.28 \times$	$7^{(0,3)}$			Coup-4	3	$1.61 \times$	10
		Coup-8	730	$1.71 \times$	$7^{(0,3)}$			Coup-8	3	$1.61 \times$	10
		Coup-16	607	$2.05 \times$	$7^{(0,3)}$			Coup-16	3	$1.80 \times$	10
		G-TD	325	$3.83 \times$	$6^{(4,0)}$			G-TD	2	$3.38 \times$	10
		G-Coup	787	$1.58 \times$	10			G-Coup	2	$2.78 \times$	10
70	4	DPA-TS	2144	$0.03 \times$	$6^{(0,4)}$	70	5	DPA-TS	—	—	$0^{(0,10)}$
		DPA-A	2130	$0.03 \times$	$7^{(0,3)}$			DPA-A	—	—	$0^{(0,10)}$
		Coup	70	$1.00 \times$	10			Coup	768	$1.00 \times$	$8^{(0,2)}$
		Coup-4	33	$2.12 \times$	10			Coup-4	728	$1.05 \times$	$9^{(0,1)}$
		Coup-8	27	$2.64 \times$	10			Coup-8	539	$1.42 \times$	$9^{(0,1)}$
		Coup-16	23	$3.08 \times$	10			Coup-16	649	$1.18 \times$	10
		G-TD	9	$8.41 \times$	10			G-TD	182	$4.22 \times$	$9^{(1,0)}$
		G-Coup	9	$8.28 \times$	10			G-Coup	291	$2.64 \times$	10
70	6	DPA-TS	—	—	$0^{(0,10)}$	70	7	DPA-TS	—	—	$0^{(0,10)}$
		DPA-A	—	—	$0^{(0,10)}$			DPA-A	—	—	$0^{(0,10)}$
		Coup	1549	$1.00 \times$	$5^{(0,5)}$			Coup	1650	$1.00 \times$	$3^{(0,7)}$
		Coup-4	1326	$1.17 \times$	$7^{(0,3)}$			Coup-4	1391	$1.19 \times$	$4^{(0,6)}$
		Coup-8	1008	$1.54 \times$	$7^{(0,3)}$			Coup-8	1757	$0.94 \times$	$6^{(0,4)}$
		Coup-16	862	$1.80 \times$	$7^{(0,3)}$			Coup-16	1708	$0.97 \times$	$7^{(0,3)}$
		G-TD	335	$4.63 \times$	$6^{(4,0)}$			G-TD	372	$4.44 \times$	$3^{(7,0)}$
		G-Coup	1010	$1.53 \times$	10			G-Coup	907	$1.82 \times$	$6^{(3,1)}$

Table 3: MOSPP Pareto Enumeration Runtime Comparison Across Enumeration Methods

N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$	N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$
150	3	DPA-TS	11	$1.12\times$	10	150	4	DPA-TS	486	$0.03\times$	10
		DPA-A	8	$1.59\times$	10			DPA-A	420	$0.04\times$	10
		Coup	12	$1.00\times$	10			Coup	16	$1.00\times$	10
		Coup-4	8	$1.50\times$	10			Coup-4	10	$1.54\times$	10
		Coup-8	8	$1.50\times$	10			Coup-8	9	$1.84\times$	10
		Coup-16	8	$1.51\times$	10			Coup-16	7	$2.16\times$	10
		G-TD	7	$1.80\times$	10			G-TD	6	$2.74\times$	10
		G-Coup	5	$2.44\times$	$9^{(1,0)}$			G-Coup	6	$2.94\times$	$9^{(1,0)}$
150	5	DPA-TS	1929	$0.02\times$	$5^{(0,5)}$	150	6	DPA-TS	1052	$0.04\times$	$1^{(0,9)}$
		DPA-A	1807	$0.02\times$	$5^{(0,5)}$			DPA-A	1085	$0.04\times$	$1^{(0,9)}$
		Coup	34	$1.00\times$	10			Coup	47	$1.00\times$	10
		Coup-4	20	$1.72\times$	10			Coup-4	26	$1.86\times$	10
		Coup-8	18	$1.89\times$	10			Coup-8	22	$2.16\times$	10
		Coup-16	16	$2.11\times$	10			Coup-16	21	$2.32\times$	10
		G-TD	10	$3.35\times$	10			G-TD	6	$8.42\times$	10
		G-Coup	11	$3.17\times$	10			G-Coup	7	$6.76\times$	10
150	7	DPA-TS	–	–	$0^{(0,10)}$	200	3	DPA-TS	32	$5.15\times$	10
		DPA-A	–	–	$0^{(0,10)}$			DPA-A	22	$7.53\times$	10
		Coup	184	$1.00\times$	10			Coup	162	$1.00\times$	$1^{(9,0)}$
		Coup-4	84	$2.20\times$	10			Coup-4	93	$1.75\times$	$1^{(9,0)}$
		Coup-8	71	$2.61\times$	10			Coup-8	87	$1.86\times$	$1^{(9,0)}$
		Coup-16	96	$1.91\times$	10			Coup-16	72	$2.27\times$	$1^{(8,1)}$
		G-TD	9	$20.88\times$	10			G-TD	–	–	$0^{(10,0)}$
		G-Coup	17	$10.98\times$	10			G-Coup	–	–	$0^{(10,0)}$
200	4	DPA-TS	913	$0.20\times$	10						
		DPA-A	793	$0.23\times$	10						
		Coup	186	$1.00\times$	$1^{(9,0)}$						
		Coup-4	80	$2.34\times$	$1^{(9,0)}$						
		Coup-8	63	$2.97\times$	$1^{(9,0)}$						
		Coup-16	45	$4.12\times$	$1^{(9,0)}$						
		G-TD	–	–	$0^{(10,0)}$						
		G-Coup	–	–	$0^{(10,0)}$						

Table 4: MOTSP Pareto Enumeration Runtime Comparison Across Enumeration Methods

N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$	N	K	Method	Time (S)	Speed-up	$S^{(M,T)}$
15	4	DPA-TS	–	–	$0^{(0,10)}$	15	5	DPA-TS	–	–	$0^{(0,10)}$
		DPA-A	–	–	$0^{(0,10)}$			DPA-A	–	–	$0^{(0,10)}$
		Coup	23	$1.00\times$	10			Coup	335	$1.00\times$	10
		Coup-4	7	$3.47\times$	10			Coup-4	115	$2.92\times$	10
		Coup-8	5	$5.06\times$	10			Coup-8	82	$4.11\times$	10
		Coup-16	3	$7.83\times$	10			Coup-16	67	$5.05\times$	10
		G-TD	3	$7.50\times$	10			G-TD	36	$9.44\times$	$9^{(1,0)}$
		G-Coup	3	$8.03\times$	10			G-Coup	48	$7.04\times$	10
15	6	DPA-TS	–	–	$0^{(0,10)}$	15	7	DPA-TS	–	–	$0^{(0,10)}$
		DPA-A	–	–	$0^{(0,10)}$			DPA-A	–	–	$0^{(0,10)}$
		Coup	1623	$1.00\times$	$9^{(0,1)}$			Coup	3544	$1.00\times$	$1^{(0,9)}$
		Coup-4	870	$1.87\times$	10			Coup-4	2508	$1.41\times$	$3^{(0,7)}$
		Coup-8	711	$2.28\times$	10			Coup-8	2123	$1.67\times$	$3^{(0,7)}$
		Coup-16	641	$2.53\times$	10			Coup-16	1930	$1.84\times$	$3^{(0,7)}$
		G-TD	–	–	$0^{(10,0)}$			G-TD	–	–	$0^{(10,0)}$
		G-Coup	369	$4.40\times$	10			G-Coup	1828	$1.94\times$	$6^{(3,1)}$